# FacePass Android  SDK User Guide V3.8.0&&V3.8.2

This document aims to introduce the installation of FacePass Android and some common operations to help users get started quickly

Document directory structure：



---

## A. SDK deployment

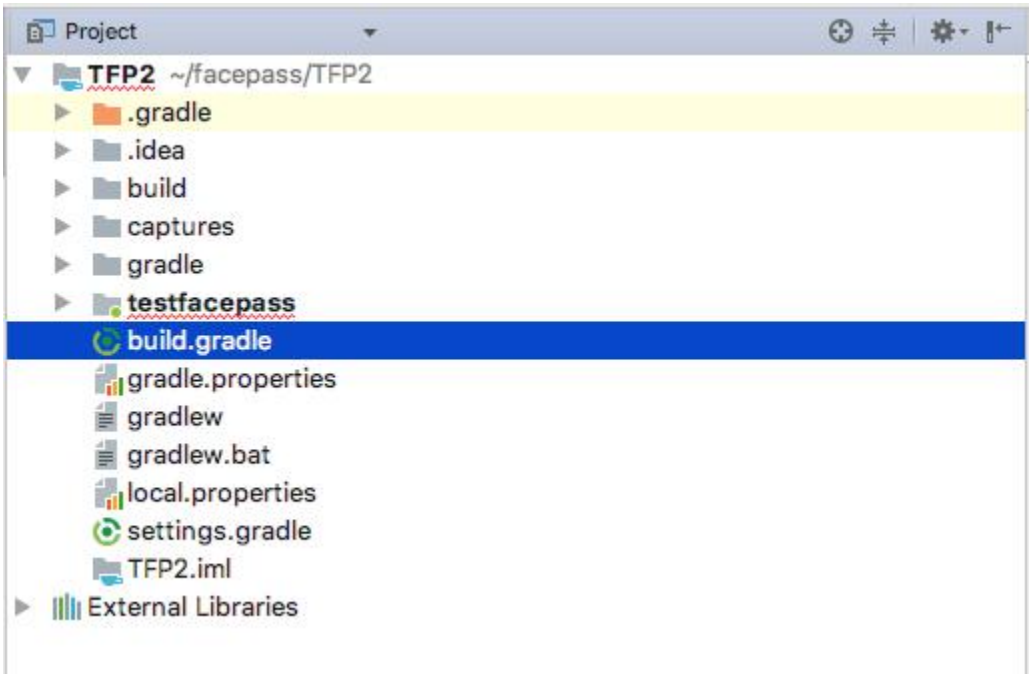
The entire client SDK consists of two parts: FacePassAndroidSDK-release.aar, an aar file and multiple .bin model files. Use the SDK to develop Android applications according to the following steps (using Android Studio as an example):
1. Put the aar file into the libs directory of the Android application to be developed (testfacepass/libs in the figure), put the model file (*.bin) into the assets directory, if there is no directory, please create a new one in the corresponding location.

2.1. Modify the configuration file build.gradle under the project root directory and add under allprojects.repositories:

```
flatDir { dirs 'libs' }
```



Branch: Shenzhen Bio Technology Co., Ltd.
Headquarter: Chongqing Huifan Technology Co., Ltd






Branch: Shenzhen Bio Technology Co., Ltd.
Headquarter: Chongqing Huifan Technology Co., Ltd

MyApplication ×

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.

buildscript {
    repositories {
        jcenter()
    }
    dependencies {
        classpath 'com.android.tools.build:gradle:2.3.3'

        // NOTE: Do not place your application dependencies here; they belong
        // in the individual module build.gradle files
    }
}

allprojects {
    repositories {
        jcenter()
        flatDir {
            dirs 'libs'
        }
    }
}

task clean(type: Delete) {
    delete rootProject.buildDir
}
```

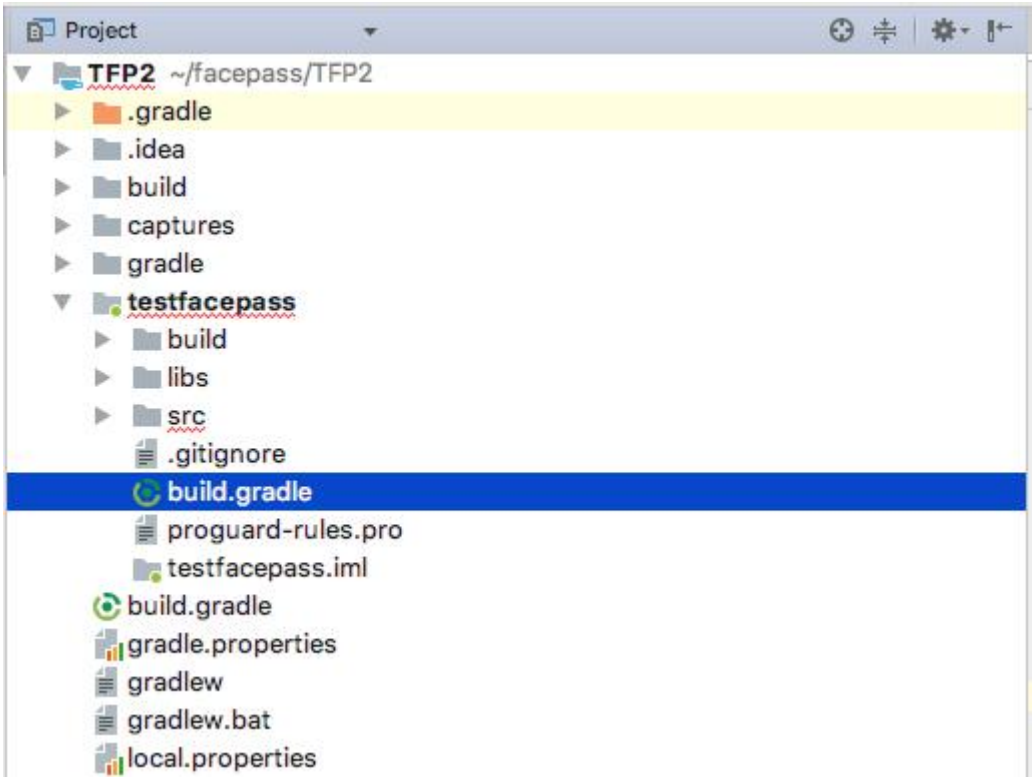
after add it, as below：



Branch: Shenzhen Bio Technology Co., Ltd.
Headquarter: Chongqing Huifan Technology Co., Ltd

1. Modify the configuration file build.gradle under the project app directory (testfacepass in the figure), and add

```
compile(name: 'FacePassAndroidSDK-release', ext: 'aar')
```



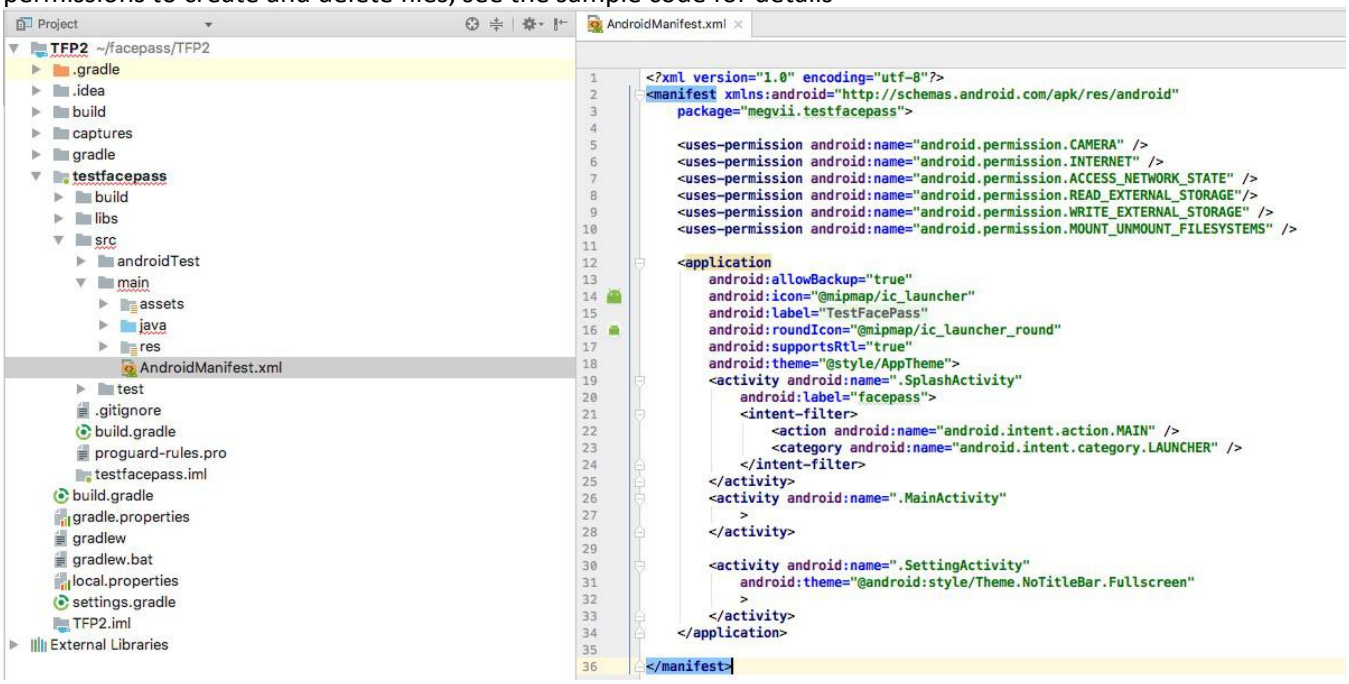

under dependencies:：

```
apply plugin: 'com.android.application'

android {
    compileSdkVersion 25
    buildToolsVersion "26.0.0"

    defaultConfig {
        applicationId "megvii.testfacepass"
        minSdkVersion 16
        targetSdkVersion 25
        versionCode 1
        versionName "1.0"
        multiDexEnabled true
        testInstrumentationRunner "android.support.test.runner.AndroidJUnitRunner"

    }
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android.txt'), 'proguard-rules.pro'
        }
    }
    packagingOptions {
        exclude 'META-INF/DEPENDENCIES'
        exclude 'META-INF/NOTICE'
        exclude 'META-INF/NOTICE.txt'
        exclude 'META-INF/LICENSE'
        exclude 'META-INF/LICENSE.txt'
        exclude 'META-INF/maven/org.java-websocket/Java-WebSocket/pom.xml'
    }
}


dependencies {
    compile fileTree(dir: 'libs', include: ['*.jar'])
    compile(name: 'FacePassAndroidSDK-release', ext: 'aar')
    androidTestCompile('com.android.support.test.espresso:espresso-core:2.2.2', {
        exclude group: 'com.android.support', module: 'support-annotations'
    })
    compile('org.apache.httpcomponents:httpmime:4.3') {
        exclude module: "httpclient"
    }
    compile 'com.android.volley:volley:1.0.0'
    compile 'com.jakewharton:butterknife:8.5.1'
    compile 'org.apache.httpcomponents:httpcore:4.3.3'
    compile 'com.android.support:appcompat-v7:25.3.1'
    compile 'com.android.support.constraint:constraint-layout:1.0.2'
    testCompile 'junit:junit:4.12'
    annotationProcessor 'com.jakewharton:butterknife-compiler:8.5.1'
}

```

2.Modify the AndroidManifest.xml file and add：

```
<uses-permission android:name="android.permission.CAMERA" />
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.MOUNT_UNMOUNT_FILESYSTEMS" />
```

Note: In the system after Android 6.0, these permissions need to be added in other ways. The required permissions include camera permissions, network permissions, access to network status, permissions to read and write files, permissions to create and delete files, see the sample code for details




5. Use SDK for development.

## B.Quick build steps

Please read the "FacePass Client SDK Document". This document only introduces the core logic and does not describe the specific API.

### 1. SDK initialization example

```
/* */
if (!hasPermission()) { requestPermission();
}

/* */
FacePass AndroidMegSafe

/* SDK */ FacePassHandler.initSDK(getApplicationContext());

/* SDKSDK Handler */ FacePassHandler mFacePassHandler; new Thread()
{
    @Override
    public void run() {
        while (isFinishing()) {
            /* SDK */
            if (FacePassHandler.isAvailable()) { FacePassConfig config;
                try {
```

```
                    /* CPU */
                    config = new FacePassConfig();
                    config.poseBlurModel = poseBlurModel;
                    config.livenessModel = livenessModel;
                    config.searchModel = searchModel;
                    config.detectModel = detectModel;
                    config.detectRectModel = detectRectModel;
                    config.landmarkModel = landmarkModel;
                    config.smileModel = smileModel;
                    config.ageGenderModel = ageGenderModel;
                    config.occlusionFilterModel = occlusionFilterModel;

                    config.livenessEnabled = true;
                    config.rgbIrLivenessEnabled = false;
                    config.smileEnabled = true;
                    config.maxFaceEnabled = true;
                    config.occlusionFilterEnabled = true;

                    config.searchThreshold = searchThreshold;
                    config.livenessThreshold = livenessThreshold;
                    config.faceMinThreshold = faceMinThreshold;
                    config.poseThreshold = new FacePassPose(roll, pitch, yaw);
                    config.blurThreshold = blurThreshold;
                    config.lowBrightnessThreshold = lowBrightnessThreshold;
                    config.highBrightnessThreshold = highBrightnessThreshold;
                    config.brightnessSTDThreshold = brightnessSTDThreshold

                    config.retryCount = retryCount;
                    config.rotation = rotation;
                    config.fileRootPath = fileRootPath;

                    /* SDK */
                    mFacePassHandler= new FacePassHandler(config);
                } catch (FacePassException e)
                    {e.printStackTrace(); return;
                }
                return;
            }
            try {
                /* SDK */
                sleep(500);
            } catch (InterruptedException e)
                { e.printStackTrace();
            }
        }
    }
}.start();
```



Branch: Shenzhen Bio Technology Co., Ltd.
Headquarter: Chongqing Huifan Technology Co., Ltd

## 2.Create base library

Create the base library through the createLocalGroup(groupName) of FacePassHandler. Through the getLocalGroups() of FacePassHandler, you can view all local base libraries.

See specifically:

```
/* Group */
private static final String group_name = "face-pass-test-5";
/* */
String[] localGroups = mFacePassHandler.getLocalGroups();
/* */
boolean isSuccess = false; try {
    isSuccess = mFacePassHandler.createLocalGroup(groupName);
} catch (FacePassException e)
    { e.printStackTrace();
}
```



Branch: Shenzhen Bio Technology Co., Ltd.
Headquarter: Chongqing Huifan Technology Co., Ltd

**3.Add a face in the client end**

FacePass can insert bitmap into the local base library. You can freely choose the picture source. The return value contains facetoken.

See specifically:

```
try {
    FacePassAddFaceResult result = mFacePassHandler.addFace(bitmap); if (result !=
    null) {
        if (result.result == 0) { toast("add face successfully");
            faceTokenEt.setText(new String(result.faceToken));
        } else if (result.result == 1) { toast("no face ");
        } else {
            toast("quality problem");
        }
    }
} catch (FacePassException e)
    { e.printStackTrace(); toast(e.getMessage());
}
```

**4.Bind base library and face**

Bind group_name to the facetoken returned above. The return value is a Boolean value, which represents whether the binding is successful.

See specifically:

```
try {
    boolean b = mFacePassHandler.bindGroup(groupName,faceToken); String result = b ?
    "success " : "failed";
    toast("bind " + result);
} catch (Exception e) { e.printStackTrace();
    toast(e.getMessage());
}
```

**5.Add camera frame**

The client generates a FacePassImage object from the camera frame and adds it to the feedFrame.

After the client completes various initializations, it needs to generate a FacePassImage object according to the image code stream from the return function of the camera, and then call the feedFrame function of the FacePassHandler and return the FacePassDetectionResult.

See specifically:

```
/* SDKFacePassImage */
FacePassImage image = newFacePassImage(cameraPreviewData.nv21Data, cameraPreviewData.width,
    cameraPreviewData.height,
    FacePassImageRotation.DEG0,
    FacePassImageType.NV21);

/* FacePassImageSDK */
FacePassDetectionResult detectionResult = mFacePassHandler.feedFrame(image);
```

**6.Call the Recognize function to process the result**

According to the message array data of FacePassDetectionResult returned by the feedFrame interface, FacePassHandler calls the mFacePassHandler.recognize interface for recognition, and the group_name needs to be passed in. Since recognize is a time-consuming operation, a separate thread needs to be opened to process the data. It is recommended to use a blocking queue to maintain the data to be processed, and obtain the data to be processed from the queue through the RecognizeThread thread training.

**See specifically:**

```
/*recognize*/
FacePassDetectionResult detectionResult = mFacePassHandler.feedFrame(image);
/*messageresultrecognize*/
if (detectionResult != null && detectionResult.message.length != 0) {
    FacePassRecognitionResult[] recognizeResult = mFacePassHandler.recognize(group_name, detecti
message);
    if (recognizeResult != null && recognizeResult.length > 0) {
        for (FacePassRecognitionResult result : recognizeResult) {
            String faceToken = new String(result.faceToken);
            if (FacePassRecognitionResultType.RECOG_OK == result.facePassRecognitionResultTy
            /* facetoken */
                getFaceImageByFaceToken(result.trackId, faceToken);
             }

        } catch (InterruptedException e) {
            e.printStackTrace();
        } catch (FacePassException e) {
            e.printStackTrace();
        }
    }
  }
}
```

**7.Exit the application and release resources**

To exit the application, you need to release the SDK Handler instance. See specifically:

```
@Override
protected void onDestroy() {
    if (mFacePassHandler != null) { mFacePassHandler.release();
    }
    super.onDestroy();
}
```

At this point, a complete pure face recognition operation is complete!